Assignment 1 - Git & GitHub Fundamentals
Session 1 – Group B - Individual Lab Work

- ***Deadline:*** Tuesday, October 13, 2026 at 7:00 PM
- ***Submission:*** GitHub repository URL in the shared Excel sheet
- ***Report:*** PDF committed inside your GitHub repository
- ***Work:*** Individual

Objective

Practice the main Git/GitHub workflow from Session 1: initialize a repository, manage files, create commits, use branches, connect to GitHub, synchronize changes, inspect history, and solve one simple Git issue.

Requirements

- Use the provided STARTER files and create a project named git-session1-<your-github-username>.
- Create at least 5 meaningful commits across at least 3 files.
- Use meaningful commit messages.
- Create a README.md and a .gitignore.
- Your final repository must contain the completed report PDF.

Tasks

1. Setup

- Initialize the project as a Git repository.
- Ignore local.env and normal .log files.
- Keep important.log visible to Git.
- Create a README.md with your name, GitHub username, short description, technologies, one code block, one link, and the provided image.

2. Commits and Staging

- Build at least 5 logical commits.
- Modify two tracked files at once, but stage and commit only one. Leave the second file modified.

3. Branch

- Create a branch for an About feature.

- Commit the feature on that branch, return to main, then merge it into main.

4. GitHub

- Create a new empty GitHub repository.
- Connect your local repository and publish main.

5. Fetch / Pull / Diff / Log

- Edit README.md directly on GitHub.
- Fetch first and verify that the remote is ahead without changing local files.
- Pull the change locally.
- Show an unstaged diff, a staged diff, and your commit history.

Report

Use the provided report template, keep explanations short, and export it to PDF. Use screenshots only when they prove an important state. For example:

- final repository/ignore state.
- rendered README.
- compact commit history.
- selective staging.
- branch/merge history.
- local main behind origin/main after fetch.
- Troubleshooting screenshot only if useful.

Fair Use of AI

Write at least one paragraph in your report. If you used AI, explain what you used it for, what kind of prompts/questions you asked, what you accepted or changed, and how you verified the answer before using it.

You are responsible for understanding every Git command used in your submission.

Submission

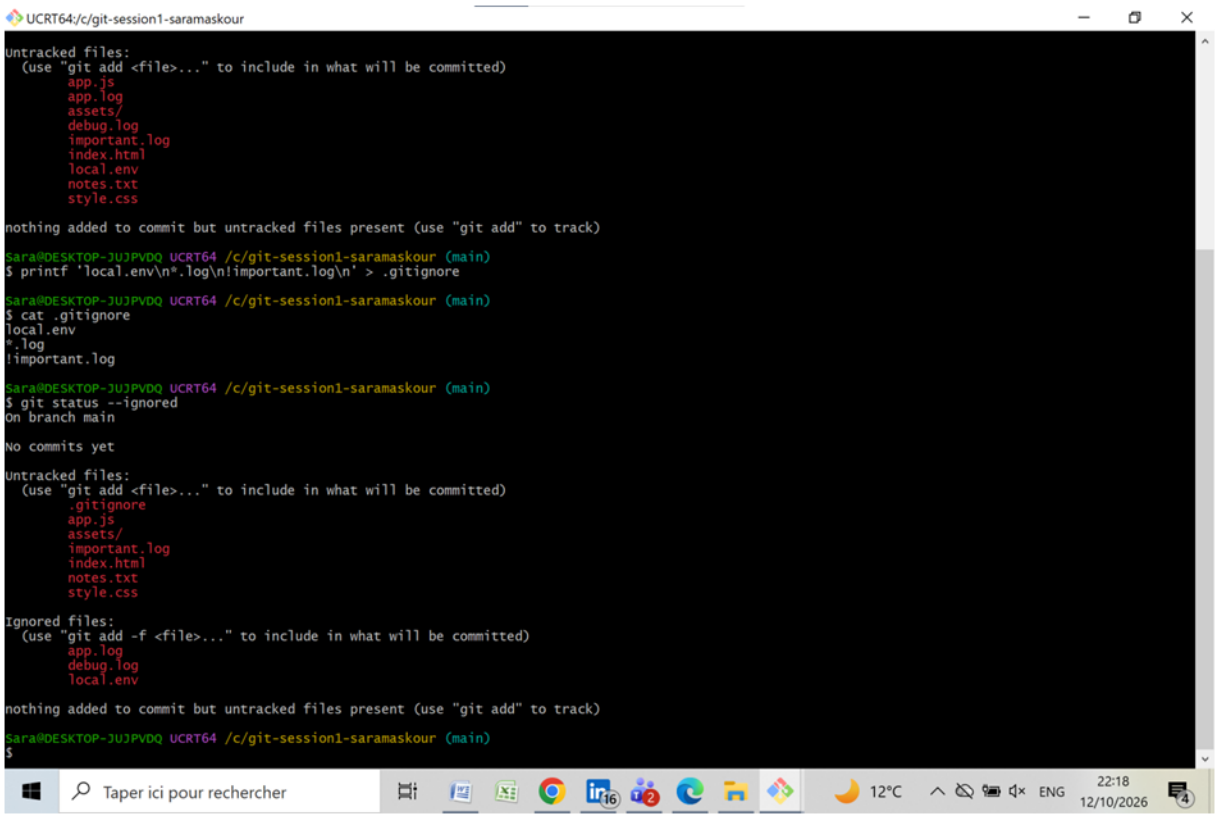
- Commit the final report PDF inside the GitHub repository.
- Make sure the repository is accessible.
- Paste the GitHub repository URL in the shared Excel sheet before the deadline.

Assignment 1 - Git & GitHub Fundamentals
Report Template

- ***Full Name:*** Sara Maskour
- ***Student ID:*** 159349
- ***Course Code & Section:*** CSC 3326 02
- ***GitHub Username:*** saramaskour

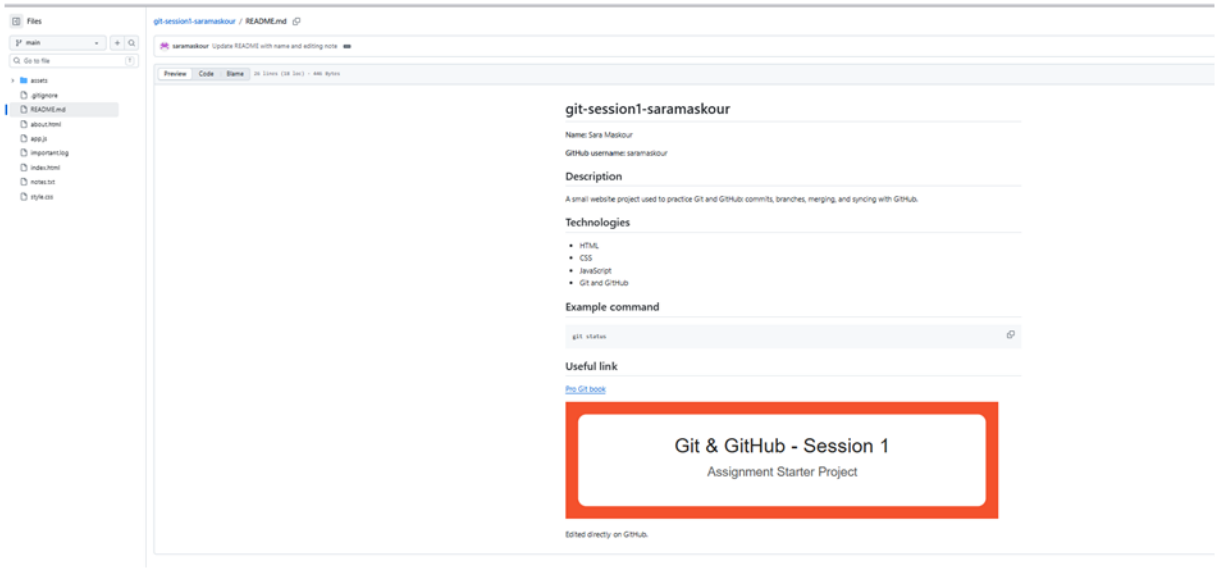
1. Setup Setup

I initialized a Git repository for the assignment and used the provided starter files. I created a `.gitignore` file to ignore `local.env` and regular `.log` files while keeping `important.log` visible to Git. I also created a README.md file containing my project information and the required content.



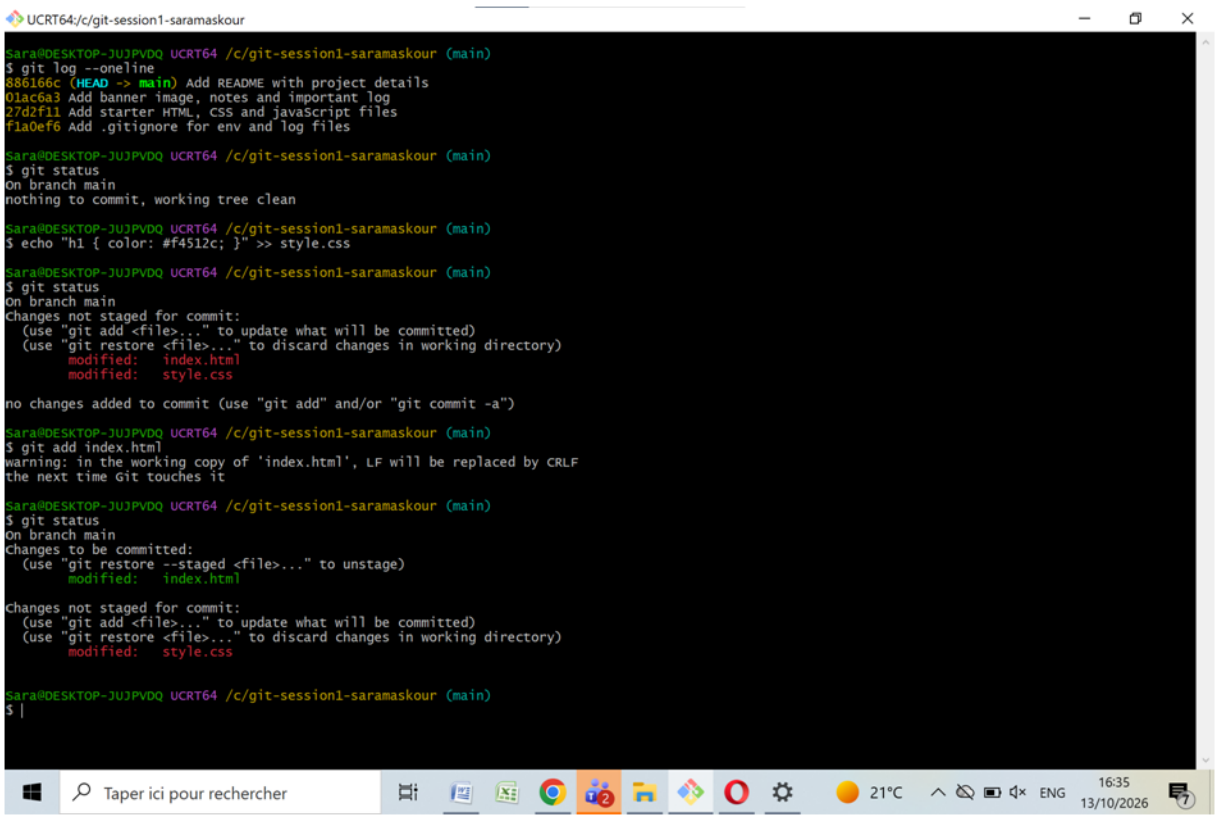
2. & Commit History History

I created a README.md file containing my name, GitHub username, project description, technologies, a code example, a link, and the provided image. I organized my work into meaningful commits so that the changes to my project could be tracked through Git history.



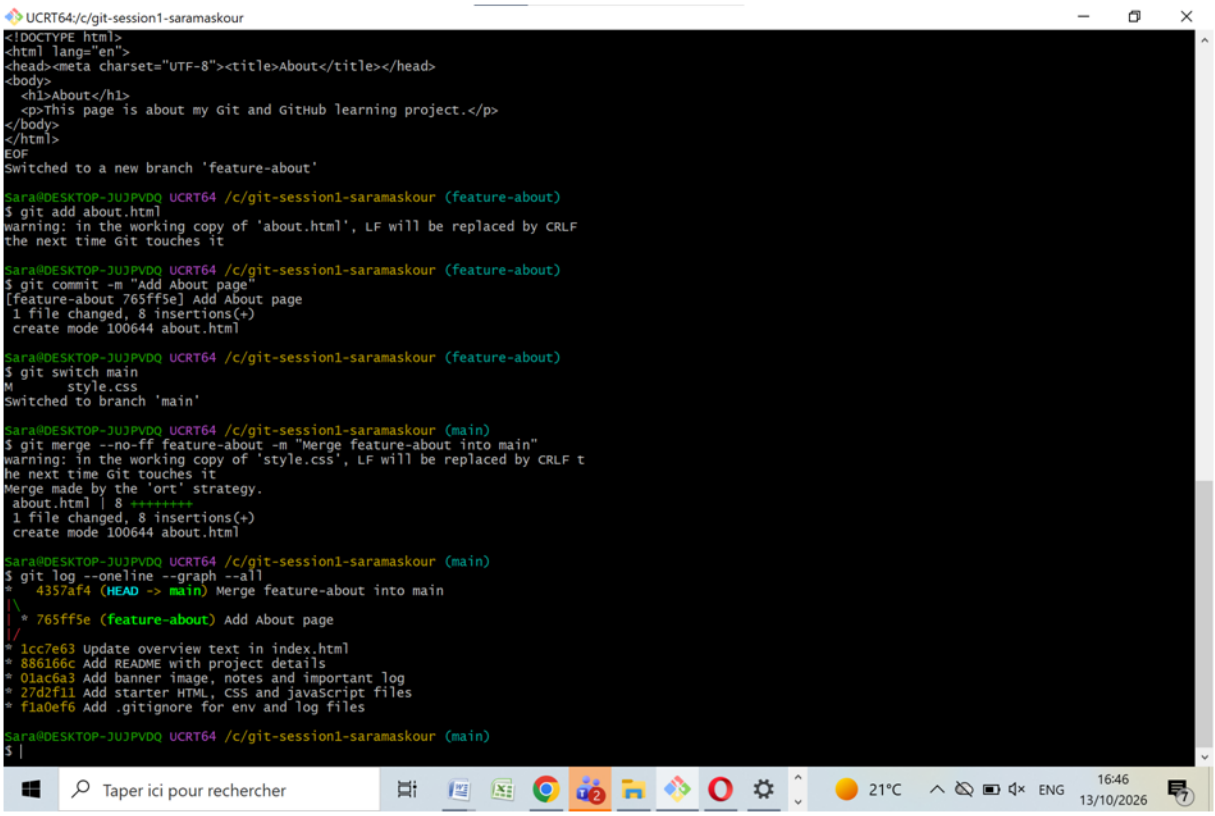
3. Staging Staging

I modified two tracked files at the same time but staged and committed only one of them. The second file remained modified but unstaged. This helped me understand how Git allows me to select which changes to include in a commit.



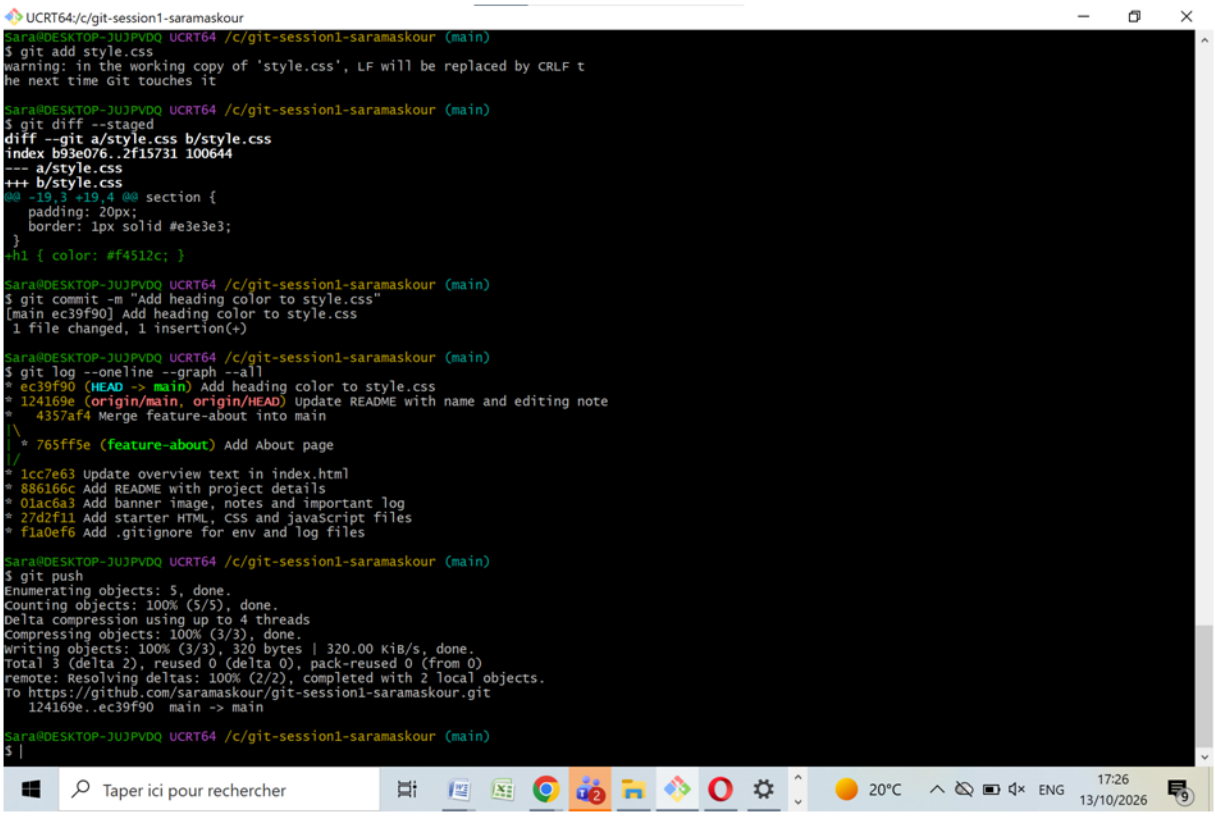
4. & Merge Merge

I created a separate branch to implement an About feature. I committed the feature on that branch, returned to the main branch, and merged the feature into main. This allowed me to practice developing a feature separately before integrating it into the main project.



5. Changes Changes

A commit saves a snapshot of changes in the local repository, while a push sends local commits to the remote GitHub repository. Fetch retrieves information about remote changes without updating my working files. Pull brings remote changes into my local branch. I edited the README.md file on GitHub, fetched the changes first, checked the remote status, and then pulled the changes into my local repository.



6. of Alof AI

I used Claude / ChatGPT as a step-by-step guide to help me understand Git and GitHub. I asked questions about Git commands, repository setup, staging, commits, branches, merging, pushing, fetching, and pulling. I typed and executed the commands myself, checked the results using commands such as `git status` and `git log`, and verified the repository on GitHub. I used the explanations to understand the workflow rather than submitting commands without understanding their purpose.

7. & Reflection Reflection

8. Git documentation: https://git-scm.com/doc

9. GitHub documentation: https://docs.github.com/

10. AI assistance: Chat GPT

The most challenging concept for me was understanding the difference between the working directory, staging area, and commit history. If Git behaves unexpectedly, I would first check `git status` to understand the current state of my files and repository before deciding what to do next.